

Guida d'uso

Riprodurre il dimero senza rumore (Parte 1) in locale

Pacchetto riproducibile — Tesi triennale, Samuele Schiavi

Relatore: Prof. Alessandro Chiesa

Questo pacchetto contiene tutto il codice necessario per riprodurre, da zero, la Parte 1 (sistema chiuso, nessun rumore) del lavoro sul dimero: diagonalizzazione esatta, confronto fra ansatz VQE, evoluzione di Trotter e correlatori dinamici. E' il pacchetto *fratello* di `pacchetto_dimero_rumoroso_riproducibile` (Parte 2) — questo copre solo il sistema chiuso, non il rumore. Ogni modulo e ogni funzione sono commentati esplicitando sia il loro ruolo all'interno del proprio file sia il loro ruolo nell'insieme della pipeline.

Indice

1	Struttura del pacchetto	1
1.1	I 9 moduli di codice/	1
2	Prerequisiti	2
3	Il modo più semplice: un solo comando	2
4	Passo per passo, con la mappa esatta delle dipendenze	2
4.1	Onda 1 — nessuna dipendenza (possono girare in qualunque ordine, anche in parallelo)	2
4.2	Onda 2 — dipende dall'Onda 1	4
4.3	In sintesi: un solo ordine realmente obbligato	4
4.4	Tutti i comandi, in un unico blocco (nell'ordine di <code>esegui_tutto.py</code>)	4
5	Un risultato genuino scoperto riproducendo la figura di confronto	5
6	Come sapere se un numero è quello giusto	5
7	Relazione con il pacchetto rumoroso	5

1 Struttura del pacchetto

<code>codice/</code>	9 moduli Python (mattoni + confronto ansatz + Trotter + correlatori)
<code>dati/</code>	dati intermedi (<code>.npz</code>) generati da <code>01_</code>
<code>figure/</code>	le 4 figure finali, in PDF e PNG
<code>genera_figure/</code>	4 script, uno per figura
<code>01_genera_dati_vqe_ideale.py</code>	unico script di generazione dati
<code>esegui_tutto.py</code>	esegue tutto in un solo comando

I 9 moduli di codice/

File	Cosa fa
<code>dimer_exact.py</code>	Hamiltoniana e diagonalizzazione esatta — il “mattoncino zero”
<code>vqe_test2.py</code>	ansatz PMA-2q.3 e ottimizzazione VQE al punto di lavoro “test 2”
<code>ansatz_dimero.py</code>	libreria di varianti di ansatz (RBS, HA, PMA a diverse parametrizzazioni)

File	Cosa fa
<code>vqe_dimer.py</code>	confronto sistematico HA vs PMA su tutto lo sweep in B/J
<code>trotter_dimero.py</code>	evoluzione di Trotter per decomposizione esplicita in gate nativi
<code>circuito_correlazioni_dimero.py</code>	circuito di Hadamard test per i correlatori dinamici
<code>noise_model_dimero.py</code>	incluso solo per la costante BASIS_GATES (vedi nota sotto)
<code>stile.py</code>	stile grafico condiviso da tutti gli script di figura
<code>validate_ansatz_params_dimero.py</code>	controllo obbligatorio: preparazione VQE vs ampiezze esatte nei correlatori
<code>validate_circuito_correlazioni_dimero.py</code>	controllo obbligatorio: correlatori contro riferimento classico, convergenza di Trotter

Una dipendenza reale verso il pacchetto rumoroso

`validate_ansatz_params_dimero.py` importa **BASIS_GATES** da `noise_model_dimero.py` — un modulo concettualmente di Parte 2. E' incluso qui **solo** per quella costante (la base di gate $\{rz, sx, x, cx\}$ usata per transpilare), non per la sua funzionalità di modellazione del rumore, mai usata in questo pacchetto.

2 Prerequisiti

Stessi di `pacchetto_dimero_rumoroso_riproducibile`:

Python	3.12.3
qiskit	2.5.2
qiskit-aer	0.17.2
numpy	2.4.4
scipy	1.17.1
matplotlib	3.10.8

Nessuna installazione LaTeX richiesta

Le figure sono generate direttamente in PDF/PNG da matplotlib.

3 Il modo più semplice: un solo comando

```
cd pacchetto_dimero_riproducibile
python3 esegui_tutto.py
```

Esegue in ordine: generazione dati (1 script), validazioni (2 script), tutte le 4 figure. Si ferma al primo errore.

Tempo stimato

1–2 minuti. La fase più lenta è la figura di confronto ansatz (circa 30 secondi per 80 ottimizzazioni VQE in tutto).

Se tutto va a buon fine, l'ultima riga stampata è:

```
TUTTO COMPLETATO. 4 figure in figure/, dati in dati/.
```

4 Passo per passo, con la mappa esatta delle dipendenze

Questa sezione non è solo un elenco di comandi: per ogni script indica cosa prende in ingresso, cosa produce (file e contenuto), e chi consuma quel prodotto più avanti nella catena. La buona notizia per questo pacchetto: la catena è quasi piatta. Un solo script produce dati; tre dei quattro

script rimanenti (una validazione, tutte le figure) **non hanno bisogno** di quel file — calcolano da soli tutto ciò che serve, in isolamento.

Onda 1 — nessuna dipendenza (possono girare in qualunque ordine, anche in parallelo)

01_genera_dati_vqe_ideale.py

Ingresso: nessuno (parte da zero).

Uscita: dati/ground_state_test2.npz — contiene b , J , D (il punto di lavoro), $E0_exact$, $psi0_exact$ (benchmark esatto), vqe_params (i 3 parametri ottimali dell'ansatz PMA-2q.3), $fidelity$, E_vqe .

Chi lo consuma: `validate_ansatz_params_dimero.py` (Onda 2, unico consumatore in questo pacchetto). Nota: è lo stesso file prodotto in `pacchetto_dimero_rumoroso_riproducibile` — se lo hai già generato là, puoi copiarlo qui invece di rieseguire lo script.

Valore atteso: energia ≈ -3.57321145 , fedeltà ≈ 1.0000000000 .

Comando (dalla radice del pacchetto):

```
python3 01_genera_dati_vqe_ideale.py
```

validate_circuito_correlazioni_dimero.py

Ingresso: nessuno — ricalcola da solo lo stato fondamentale via `ground_state()` di `circuito_correlazioni_dimero.py`.

Uscita: nessun file, solo un resoconto a schermo.

Chi lo consuma: nessuno (è un controllo terminale della catena).

Comando (dalla radice del pacchetto, nota il `cd`):

```
cd codice
python3 validate_circuito_correlazioni_dimero.py
cd ..
```

genera_fig_spettro_esatto.py

Ingresso: nessuno — diagonalizza da zero (nessun `.npz` richiesto).

Uscita: figure/fig_spettro_dimero.pdf (+.png).

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

```
python3 genera_figure/genera_fig_spettro_esatto.py
```

genera_fig_confronto_ansatz.py

Ingresso: nessuno — esegue da zero 80 ottimizzazioni VQE (4 configurazioni $\times$ 20 punti in b).

Uscita: figure/fig_confronto_ansatz_HA_PMA.pdf (+.png).

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto, ~ 30 secondi):

```
python3 genera_figure/genera_fig_confronto_ansatz.py
```

genera_fig_trotter_convergenza.py

Ingresso: nessuno — costruisce l'evoluzione via `trotter_dimero.py` da zero, a b , J , D fissati nello script.

Uscita: figure/fig_trotter_convergenza.pdf (+.png).

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

```
python3 genera_figure/genera_fig_trotter_convergenza.py
```

```
genera_fig_correlatore_convergenza.py
```

Ingresso: nessuno — ricalcola lo stato fondamentale e il riferimento classico da zero.

Uscita: figure/fig_correlatore_convergenza.pdf (+.png).

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

```
python3 genera_figure/genera_fig_correlatore_convergenza.py
```

Onda 2 — dipende dall'Onda 1

```
validate_ansatz_params_dimero.py
```

Ingresso: ground_state_test2.npz (copiato in codice/) — legge vqe_params e fidelity.

Uscita: nessun file, solo un resoconto a schermo (include il controllo 18 vs 19 CNOT).

Chi lo consuma: nessuno (controllo terminale).

Comando (dalla radice del pacchetto, dopo aver eseguito 01_):

```
cp dati/ground_state_test2.npz codice/
cd codice
python3 validate_ansatz_params_dimero.py
cd ..
```

In sintesi: un solo ordine realmente obbligato

L'unica dipendenza vera di questo pacchetto

01_genera_dati_vqe_ideale.py → validate_ansatz_params_dimero.py. Tutto il resto (l'altra validazione, tutte e 4 le figure) è indipendente e può girare prima, dopo, o senza mai eseguire 01_ — esegui_tutto.py li esegue comunque tutti in un ordine fisso solo per chiarezza di lettura del log, non perché serva.

Tutti i comandi, in un unico blocco (nell'ordine di esegui_tutto.py)

```
cd pacchetto_dimero_riproducibile
python3 01_genera_dati_vqe_ideale.py
cp dati/*.npz codice/
cd codice
python3 validate_ansatz_params_dimero.py
python3 validate_circuito_correlazioni_dimero.py
cd ..
python3 genera_figure/genera_fig_confronto_ansatz.py
python3 genera_figure/genera_fig_correlatore_convergenza.py
python3 genera_figure/genera_fig_spettro_esatto.py
python3 genera_figure/genera_fig_trotter_convergenza.py
```

Risultati chiave dei due controlli

validate_ansatz_params_dimero.py: costo in gate, preparazione esatta 18 CNOT contro preparazione VQE 19 CNOT (coincide esattamente con quanto atteso).

validate_circuito_correlazioni_dimero.py: convergenza dell'errore di Trotter sul correlatore, rapporto che tende a 2 raddoppiando N (Trotter al prim'ordine, errore $O(1/N)$).

Gli script di figura vanno eseguiti dalla **radice** del pacchetto, non da dentro genera_figure/.

5 Un risultato genuino scoperto riproducendo la figura di confronto

PMA a $K = 1$ non è sempre sufficiente

Nella figura `fig_confronto_ansatz_HA_PMA`, la riga “PMA $D = 0.2$ ” mostra un calo netto di fedeltà (minimo ≈ 0.82 , non 1.0) proprio nella zona di anticrossing ($B/J \approx 2$) — a differenza di HA, che raggiunge sempre precisione macchina. Non è un bug: con un solo parametro e uno stato iniziale scelto in modo rigido (singoletto o $|11\rangle$ a seconda di B/J), l'ansatz PMA più semplice non riesce a interpolare con continuità attraverso il crossover indotto da $D \neq 0$. E' precisamente il motivo per cui il resto del progetto usa un ansatz PMA più espressivo (PMA-2q.3, 3 parametri, che rompe la conservazione di M) invece di questa versione a un solo parametro.

6 Come sapere se un numero è quello giusto

Ogni script stampa a schermo il valore appena calcolato **accanto** al valore atteso.

Se un numero non coincide

Qualcosa nel tuo ambiente locale sta producendo un risultato diverso: fermati e confronta prima di proseguire.

7 Relazione con il pacchetto rumoroso

Stessa disciplina, stesso punto di partenza

Questo pacchetto e `pacchetto_dimero_rumoroso_riproducibile` condividono lo script `01_genera_dati_vqe_ideale.py` e tre moduli (`dimer_exact.py`, `vqe_test2.py`, `circuito_correlazioni_dimero.py`) — non per caso: la Parte 2 (rumore) parte esattamente da dove finisce questa Parte 1, riusando lo stesso punto di lavoro e la stessa preparazione VQE. Se hai già eseguito il pacchetto rumoroso, `dati/ground_state_test2.npz` è identico in entrambi.

Verificato end-to-end, non solo assemblato

L'intera pipeline è stata rieseguita da zero (cartelle `dati/` e `figure/` svuotate, poi `esegui_tutto.py` rilanciato) dopo ogni modifica sostanziale al codice — ogni numero riportato in questa guida è stato ricontrollato.